“Tree Network Analysis and Optimization System:
Tree Diameter Using Two-DFS and Tree Dynamic Programming”

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms – 3 (25CS2103E)

in

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

by

Upputuri Rohit Sai – 2520030030
P pratik kumar – 2520030034
Mvsr Hitesh – 2520039621

Team Number: 14
Section: S1

Under the Guidance of
Dr. Ch. Anuradha
Associate Professor
Department of Computer Science and Engineering



KL (Deemed to be) University, Hyderabad
2026–27

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Declaration

The Project Report entitled **"Tree Network Analysis and Optimization System: Tree Diameter Using Two-DFS and Tree Dynamic Programming"** is a record of bonafide work of **Upputuri Rohit Sai (2520030030), P pratik kumar (2520030034) and Mvsr Hitesh (2520039621)** submitted in partial fulfillment for the award of B.Tech in Computer Science and Engineering. The results embodied in this report have not been copied from any other departments/University/Institute.

Upputuri Rohit Sai – 2520030030

P pratik kumar – 2520030034

Mvsr Hitesh – 2520039621

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the mini project based report entitled **"Tree Network Analysis and Optimization System: Tree Diameter Using Two-DFS and Tree Dynamic Programming"** is a bonafide work done and submitted by **Upputuri Rohit Sai (2520030030), P pratik kumar (2520030034) and Mvsr Hitesh (2520039621)** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in the Department of Computer Science and Engineering, during the academic year **2026–27**.

Signature of the GuideSignature of the HOD
(Dr. Ch. Anuradha)

Signature of the External Examiner

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, **Dr. Ch. Anuradha**, Associate Professor, Department of Computer Science and Engineering, for the valuable guidance, continuous support, and constructive feedback offered throughout the design and implementation of this project.

We would also like to extend our thanks to **[HOD NAME REQUIRED]**, Head of the Department of Computer Science and Engineering, for providing the academic infrastructure and resources needed to carry out this work successfully. Finally, we thank KL (Deemed to be) University, Hyderabad, for giving us the opportunity to work on this Data Structures and Algorithms – 3 project.

Upputuri Rohit Sai – 2520030030

P pratik kumar – 2520030034

Mvsr Hitesh – 2520039621

Abstract

Trees are among the most widely used structures for modelling hierarchical and loop-free networks. In a communication network built as a tree, the longest path between any two nodes, known as the tree diameter, determines the worst-case number of hops that a message may have to travel. Finding this value efficiently, and verifying that it is correct, is the central problem addressed in this project.

This report presents the design and development of a **Tree Network Analysis and Optimization System** implemented in Java. The application reads the nodes and edges of a tree, stores them in an adjacency list, and validates that the input is a genuine tree, i.e., connected and free of cycles. It then displays the tree structure and computes basic properties such as the degree of each node, the number of leaf nodes, and the height of the tree with respect to the selected root.

The tree diameter is computed using two independent algorithms: the **Two-DFS (double Depth-First Search) method** and a **Tree Dynamic Programming method** that combines the two tallest child subtrees at each node. Both algorithms run in linear time, O(N), for a tree of N nodes. The system compares the results of the two approaches to cross-check correctness and interprets the diameter as the longest communication route, reporting the end-point nodes and the number of communication hops.

By integrating a graph-theoretic problem with two algorithm-design paradigms in a modular Java program, the project demonstrates how core DSA-3 concepts apply to the analysis of a simple communication network. For the six-node tree that was executed during testing, both algorithms reported a diameter of four edges ($4 \rightarrow 2 \rightarrow 1 \rightarrow 3 \rightarrow 6$), the program confirmed that the two results agree, and it suggested node 1 as the central server. Only this single case has been run so far, and the remaining planned test cases are yet to be verified.

Keywords: Tree, Tree Diameter, Depth-First Search, Tree Dynamic Programming, Adjacency List, Communication Network, Java.

Table of Contents

1. INTRODUCTION **1**
1.1 Overview 1
1.2 Motivation 1
1.3 Problem Statement 2
1.4 Objectives 2
1.5 Scope and Boundaries 3
2. BACKGROUND AND SYSTEM REQUIREMENTS **4**
2.1 Background and Fundamental Concepts 4
2.2 Functional Requirements 5
2.3 Non-Functional Requirements 6
2.4 Hardware and Software Specifications 6
3. SYSTEM DESIGN **7**
3.1 High-Level Architecture Diagram 7
3.2 Description of Each Stage 7
3.3 Data Structures Used 8
4. IMPLEMENTATION AND PSEUDOCODE **9**
4.1 Module-Wise Implementation 9
4.2 Worked Example Used to Explain the Algorithms 9
4.3 Module: Tree Construction and Validation 10
4.4 Module: Tree Analysis (Degree, Leaves, Height) 11
4.5 Module: Diameter Using Two-DFS 11
4.6 Module: Diameter Using Tree Dynamic Programming 12
4.7 Module: Comparison and Network Interpretation 12
4.8 Comparison of the Two Approaches 13
5. RESULTS **14**
5.1 Program Output Screenshots 15
5.2 Discussion of Results 15
6. CHALLENGES, LIMITATIONS AND SOLUTIONS **17**
6.1 Issues Faced During Development 17
6.2 Solutions Applied 17
6.3 Limitations of the System 17
7. CONCLUSION AND FUTURE WORK **18**
7.1 Conclusion 18
7.2 Future Work 18
7.3 Skills Learned 18
REFERENCES **19**

List of Figures

Figure 3.1: High-Level Architecture and Execution Flow of the Tree Network Analysis System

Figure 4.1: Example Tree Used for the Worked Illustration (8 nodes)

Figure 5.1: Program Start-up and Tree Input Screen

Figure 5.2: Tree Display, Degree, Leaf and Height Output

Figure 5.3: Two-DFS and Tree DP Diameter Output with Comparison

Figure 5.4: Communication Route Interpretation Output

List of Tables

Table 1.1: Summary of Project Details

Table 2.1: Hardware and Software Specifications

Table 4.1: Degree and Height Computation for the Example Tree

Table 4.2: Trace of the Two-DFS Algorithm on the Example Tree

Table 4.3: Trace of the Tree DP Algorithm on the Example Tree

Table 4.4: Complexity Summary of the Implemented Operations

Table 5.1: Test Cases and Results

1. INTRODUCTION

1.1 Overview

A **tree** is a connected, undirected graph that contains no cycles. A tree with N vertices has exactly N − 1 edges, and there is exactly one simple path between every pair of vertices. These properties make trees a natural model for hierarchical organisations, file systems, routing hierarchies, and loop-free communication networks such as a simple star-of-stars or backbone-and-branch topology.

The **Tree Network Analysis and Optimization System** is a Java console application that treats an input tree as a small communication network. Each vertex represents a device or station, and each edge represents a direct communication link. The program accepts the nodes and edges, checks that they really form a tree, stores them in an adjacency list, and analyses the structure of the network. Its principal result is the **diameter** of the tree: the number of edges on the longest path between any two vertices. In network terms, the diameter corresponds to the longest communication route, which gives the worst-case number of hops between two stations.

The diameter is computed with two different algorithms: (1) the **Two-DFS method**, which runs a depth-first search from an arbitrary node to find the farthest node and then a second search from that node, and (2) a **Tree Dynamic Programming method**, which computes subtree heights bottom-up and combines the two tallest branches at every node. Computing the same quantity in two ways lets the program cross-verify its output and lets the reader compare the two techniques.

Table 1.1: Summary of Project Details

Item	Details
Project Title	Tree Network Analysis and Optimization System
Subtitle	Tree Diameter Using Two-DFS and Tree Dynamic Programming
Course	Data Structures and Algorithms – 3 (25CS2103E)
Programming Language	Java
Core Techniques	Adjacency list, Depth-First Search, Tree Dynamic Programming
Institution	KL (Deemed to be) University, Hyderabad
Department	Computer Science and Engineering
Academic Year	2026–2027

1.2 Motivation

Many practical networks, including campus backbones, sensor-network aggregation structures, and hierarchical distribution systems, are organised as trees because a tree provides connectivity with the fewest possible links and without redundant loops. For such a network, a designer wants to know how far apart the two most distant stations are. The answer tells how long a message may take in the worst case and highlights the stretch of the network that is most critical to latency.

A brute-force approach would run a search from every node and take the maximum, which costs $O(N^2)$ time for a tree with N nodes. For large networks this is wasteful. The structure of a tree allows the same answer to be found in O(N) time, and two different linear-time algorithms exist. Implementing both gives a deeper understanding of graph traversal and dynamic programming than implementing either one alone, which is the academic motivation for this DSA-3 project.

It is important to separate the theoretical motivation from the implemented features. The sections above describe why tree analysis is valuable in general; the features that the program actually provides are listed in Sections 1.4 and 1.5 and are described in detail in Chapters 3 and 4.

1.3 Problem Statement

Given a valid tree T = (V, E) with N vertices, find the **longest simple path between any two vertices** and report its length (the diameter), computed independently by the Two-DFS method and by Tree Dynamic Programming. In addition, the system must validate the input, display the tree, and report supported structural information such as node degrees, the number of leaf nodes, and the tree height, and must present the diameter as the longest communication route together with its end-point nodes and number of hops.

Expressed formally, the diameter is $\text{diam}(T) = \max \{ \text{dist}(u, v) : u, v \in V \}$, where dist(u, v) is the number of edges on the unique path between u and v.

1.4 Objectives

1. To build a tree from user-supplied nodes and edges.
2. To validate that the input is a tree, i.e., it has $N - 1$ edges, is connected, and contains no cycles.
3. To represent the tree efficiently using an adjacency list.

4. To compute basic tree properties: degree of every node, number of leaf nodes, and height with respect to the selected root.
5. To implement the tree diameter algorithm using the Two-DFS approach.
6. To implement the tree diameter algorithm using Tree Dynamic Programming.
7. To compare the diameters produced by the two algorithms and confirm that they agree.
8. To demonstrate the practical interpretation of the diameter as the longest communication route in a network.
9. To suggest a central server node for the network, as reported in the program output.

1.5 Scope and Boundaries

The project is restricted to **unweighted, undirected trees** processed by a Java console program. Within this scope, the system handles input, validation, display, structural analysis, diameter computation with two algorithms, comparison, route interpretation and a suggested central server node.

The following are **outside the scope** of the present implementation: weighted edges, directed edges, graphical user interfaces, database storage, real-time or packet-level network simulation, and deployment on real network hardware. These are discussed only as future work in Chapter 7.

2. BACKGROUND AND SYSTEM REQUIREMENTS

2.1 Background and Fundamental Concepts

2.1.1 Graphs and Trees

A graph G = (V, E) consists of a set of vertices V and a set of edges E. A graph is **connected** if a path exists between every pair of vertices, and it is **acyclic** if it contains no cycle. A tree is a connected acyclic graph. Equivalent characterisations that the program relies on for validation are: a graph with N vertices is a tree if and only if it is connected and has exactly N − 1 edges, or if and only if it is acyclic and has exactly N − 1 edges.

2.1.2 Adjacency List

An adjacency list stores, for every vertex, the list of its neighbours. For a tree with N vertices it needs O(N) space, because the total number of entries is 2(N − 1). Iterating over the neighbours of a vertex costs time proportional to its degree, so a complete traversal of the tree takes O(N) time. This is much better than an N × N adjacency matrix, which needs $O(N^2)$ space and O(N) time just to find the neighbours of a single vertex.

2.1.3 Depth-First Search (DFS)

DFS explores a graph by moving as deep as possible along one branch before backtracking. In this project, DFS is used to compute the distance (number of edges) from a starting node to every other node. Because a tree has a unique path between any two nodes, the first time DFS reaches a node it does so along the only simple path, and the depth recorded is the exact distance. A parent argument (or visited array) prevents the search from moving back to the node it just came from.

2.1.4 Degree, Leaf Nodes and Height

The **degree** of a vertex is the number of edges incident on it. A **leaf node** is a vertex of degree one (for a tree with at least two vertices). The **height** of a rooted tree is the number of edges on the longest downward path from the root to a leaf. Height depends on the choice of root, whereas the diameter does not.

2.1.5 Tree Diameter

The diameter of a tree is the number of edges on the longest path between any two vertices. The end-points of such a path are always leaf nodes, since a path ending at a non-leaf vertex could be extended through one more neighbour. A tree may have more than one longest path, but all of them have the same length.

Correctness of Two-DFS. Start from any vertex s and let a be the vertex farthest from s. A standard result states that a is an end-point of some diameter. Running a second search from a and taking the farthest vertex b gives a path a–b of length diam(T). The argument is that in a tree with non-negative edge lengths, the farthest vertex from any starting vertex is always one end of a longest path.

2.1.6 Dynamic Programming on Trees

Dynamic programming (DP) solves a problem by combining the answers to overlapping or nested sub-problems. On a rooted tree, the natural sub-problem is a subtree. Let h(v) be the height of the subtree rooted at v, defined by h(v) = 0 for a leaf and h(v) = 1 + max h(c) over all children c of v otherwise. The longest path whose highest node is v is then the sum of the two largest values of (h(c) + 1) over the children of v (or just one value if v has a single child). The diameter is the maximum of this quantity over all vertices.

2.1.7 Time and Space Complexity

Time complexity expresses the growth of the running time with the input size, and space complexity does the same for the memory used. Both diameter algorithms in this project visit every vertex and every edge a constant number of times, so they run in O(N + E) = O(N) time and use O(N) additional memory. The depth of the recursion is O(N) in the worst case (a path-like tree), which is discussed in Chapter 6.

2.2 Functional Requirements

- **Tree Input:** The system must accept the number of nodes, the number of edges and the list of edges from the user (in the demonstrated run: 6 nodes, 5 edges and five node pairs).
- **Validation:** The system must reject input that is not a tree (wrong number of edges, invalid node numbers, self-loops, duplicate edges, cycles, or a disconnected graph.
- **Tree Display:** The system must display the tree structure, e.g., as an adjacency listing, in the terminal.
- **Tree Analysis:** The system must report the degree of each node, the number of leaf nodes, and the height of the tree with respect to the chosen root.
- **Diameter by Two-DFS:** The system must compute the diameter using two depth-first searches.
- **Diameter by Tree DP:** The system must compute the diameter using Tree Dynamic Programming.

- **Comparison:** The system must compare both results and report whether they match.
- **Network Interpretation:** The system must present the diameter as the longest communication route, with the end-point nodes and the number of hops.
- **Central Server Suggestion:** The system should suggest a central server node, as reported in the demonstrated output (node 1 for the six-node test tree).

2.3 Non-Functional Requirements

- **Performance:** Both diameter computations must run in linear time, O(N).
- **Correctness:** The two algorithms must give the same diameter for every valid tree.
- **Usability:** The console prompts and outputs should be clear and well formatted.
- **Reliability:** Invalid input should produce a clear error message rather than a crash.
- **Maintainability:** The code should be modular, with separate methods or classes for input, validation, analysis and each algorithm.

2.4 Hardware and Software Specifications

Table 2.1: Hardware and Software Specifications

Component	Specification
Operating System	Windows (as shown in the program-output screenshot); exact version not stated
Language / Platform	Java SE Development Kit (JDK 25, as shown in the program-output screenshot)
IDE / Editor	Visual Studio Code (VS Code Java launcher visible in the program-output screenshot)
Version Control	GitHub repository: https://github.com/urohitsai-264/KLH-CSE-2026-2027-14-TreeDiameter_DP
Minimum Hardware	Any system capable of running a JDK; no special hardware needed

The operating system, JDK and editor entries above are taken from the program-output screenshot; the hardware used was not recorded.

3. SYSTEM DESIGN

3.1 High-Level Architecture Diagram

The system follows a simple pipeline architecture. Data moves from user input through construction and validation of the tree, then through display and analysis, and finally to the two diameter algorithms, the comparison step, and the interpretation of the result as a communication route. Figure 3.1 shows this flow.

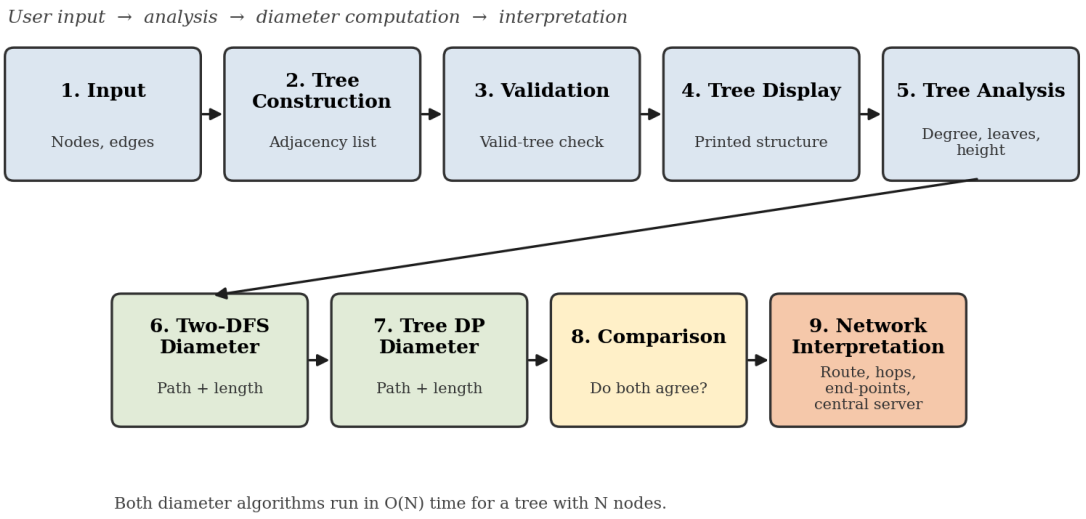


Figure 3.1: High-Level Architecture and Execution Flow of the Tree Network Analysis System

3.2 Description of Each Stage

Stage 1: Input

The user supplies the number of nodes and the edges that connect them. Each edge joins two node labels. Node labels are mapped to array indices so that the adjacency list can be indexed directly. In the demonstrated run, the labels run from 1 to 6 (one-based) and the nodes and edges are entered at the console.

Stage 2: Tree Construction

For each edge (u, v), v is added to the neighbour list of u and u to that of v, because the tree is undirected. The structure used is an array (or list) of lists, e.g., `List<List<Integer>>` or `ArrayList<Integer>[]`.

Stage 3: Validation

The program checks that the input describes a valid tree. The checks follow from the definition: the number of edges must be N − 1, every node label must be in range, and a

traversal started from one node must reach all N nodes (connectivity). A connected graph with N − 1 edges is necessarily acyclic, so these checks together guarantee a tree.

Stage 4: Tree Display

The tree structure is printed in the terminal before the analysis results (see Figure 5.2).

Stage 5: Tree Analysis

The program computes the degree of each node (the size of its neighbour list), counts the nodes with degree one as leaves, and computes the tree height from the selected root with a DFS.

Stage 6: Diameter Using Two-DFS

A first DFS from an arbitrary node finds the farthest node a. A second DFS from a finds the farthest node b and its distance, which is the diameter. The end-points a and b and the path between them are available for the interpretation stage.

Stage 7: Diameter Using Tree DP

A single post-order DFS computes the subtree height h(v) for each node. At each node, the two largest child contributions are added, and the maximum over all nodes is the diameter.

Stage 8: Comparison

The two diameter values are compared. Equal values confirm that both implementations agree; a mismatch would indicate an implementation error.

Stage 9: Network Interpretation

The diameter is reported as the longest communication route: the two end-point nodes, the number of hops (edges), and the nodes along the route. The program also prints a suggested central server node (node 1 in the demonstrated run, which is the middle node of the route 4 → 2 → 1 → 3 → 6).

3.3 Data Structures Used

Data Structure	Purpose
Adjacency list (list of lists)	Stores the undirected tree in O(N) space
Visited / parent tracking	Prevents the traversal from revisiting nodes
Distance array (int[])	Stores the distance of each node from the DFS start node (Two-DFS)
Height array / return values	Stores the subtree height h(v) (Tree DP)
Parent array (optional)	Used to reconstruct the diameter path

4. IMPLEMENTATION AND PSEUDOCODE

4.1 Module-Wise Implementation

The program is organised into modules, each responsible for one stage of the pipeline described in Chapter 3.

Module	Responsibility	Java class / method
Input & Construction	Reads N and the edges and builds the adjacency list	Main.java
Validation	Checks edge count, node range and connectivity	Main.java
Display	Prints the tree structure	Main.java
Analysis	Degree, leaf count, height	Main.java
Two-DFS Diameter	Computes the diameter with two searches	DiameterDFS.java
Tree DP Diameter	Computes the diameter from subtree heights	DiameterDP.java
Comparison & Interpretation	Compares results, prints route information and suggested central server	Comparison.java, TreeApplications.java
Main	Controls the order of execution	Main.java

4.2 Worked Example Used to Explain the Algorithms

To make the algorithms concrete, a small example tree with 8 nodes and 7 edges is used throughout this chapter. The edges are 1–2, 1–3, 2–4, 2–5, 3–6, 5–7 and 7–8. Node 1 is chosen as the root. The values in the tables below were derived by hand from the definitions and are used to explain the algorithms; they are not copied from program output.

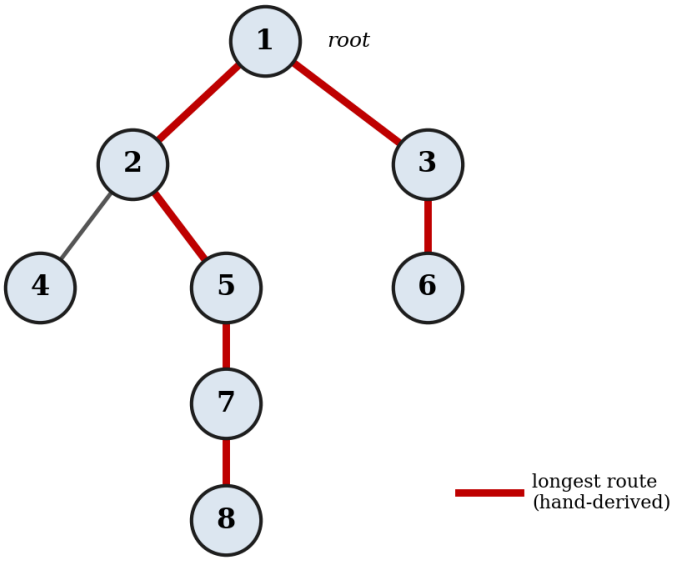


Figure 4.1: Example Tree Used for the Worked Illustration (8 nodes)

Table 4.1: Degree and Height Computation for the Example Tree

Node	Neighbours	Degree	Leaf?	Height of subtree h(v) (root = 1)
1	2, 3	2	No	4
2	1, 4, 5	3	No	3
3	1, 6	2	No	1
4	2	1	Yes	0
5	2, 7	2	No	2
6	3	1	Yes	0
7	5, 8	2	No	1
8	7	1	Yes	0

The example tree has 3 leaf nodes (4, 6 and 8) and a height of 4 when rooted at node 1.

4.3 Module: Tree Construction and Validation

The adjacency list is built from the edge list. Validation then checks the edge count, node range and connectivity.

```
ALGORITHM BuildAndValidate(N, edges)
    IF length(edges) != N - 1 THEN RETURN INVALID
    CREATE adj[1..N] as empty lists
    FOR EACH (u, v) IN edges DO
        IF u OR v is out of range OR u == v THEN RETURN INVALID
        ADD v TO adj[u]
        ADD u TO adj[v]
    visited = array of size N+1, all false
    count = DFSCount(1, visited)        // number of nodes reached
    IF count != N THEN RETURN INVALID  // disconnected or has a cycle
```

```
    RETURN adj
```

Since a connected graph with exactly N − 1 edges cannot contain a cycle, the edge-count test and the connectivity test together are sufficient to prove that the input is a tree. Time complexity is O(N).

4.4 Module: Tree Analysis (Degree, Leaves, Height)

```
ALGORITHM Analyse(adj, N, root)
    leafCount = 0
    FOR v FROM 1 TO N DO
        degree[v] = size(adj[v])
        IF degree[v] == 1 THEN leafCount = leafCount + 1
    height = Height(root, -1)
    RETURN degree[], leafCount, height

ALGORITHM Height(u, parent)
    best = 0
    FOR EACH v IN adj[u] DO
        IF v != parent THEN best = MAX(best, 1 + Height(v, u))
    RETURN best
```

4.5 Module: Diameter Using Two-DFS

The algorithm relies on the property that the farthest node from any starting node is an end of some longest path. Two traversals are therefore enough.

```
ALGORITHM FarthestFrom(src)
    dist[1..N] = -1
    DFS(src, dist, 0)               // fills dist[] with depths
    far = node with maximum dist[]
    RETURN (far, dist[far])

ALGORITHM DiameterTwoDFS(adj)
    (a, _) = FarthestFrom(1)        // 1st DFS: any start node
    (b, d) = FarthestFrom(a)        // 2nd DFS: from end-point a
    RETURN (a, b, d)                // d = diameter in edges
```

Table 4.2: Trace of the Two-DFS Algorithm on the Example Tree

Step	Start node	Distances from start node (node: distance)	Farthest node
1st DFS	1	1:0, 2:1, 3:1, 4:2, 5:2, 6:2, 7:3, 8:4	8 (distance 4)
2nd DFS	8	8:0, 7:1, 5:2, 2:3, 1:4, 4:4, 3:5, 6:6	6 (distance 6)

Hence the diameter is **6 edges**, with end-points 8 and 6, and the longest route is 8 → 7 → 5 → 2 → 1 → 3 → 6.

Complexity. Each DFS visits every node and edge once, so the total time is O(N) and the space is O(N) for the distance array and the recursion stack.

4.6 Module: Diameter Using Tree Dynamic Programming

Here the tree is rooted (at node 1 in the example). For every node u the height h(u) of its subtree is computed bottom-up. While processing u, the two largest values of (h(c) + 1) over its children c are kept, and their sum is a candidate for the diameter: it is the length of the longest path that passes through u as its highest node.

```
ALGORITHM DiameterTreeDP(adj, root)
    diameter = 0
    CALL DP(root, -1)
    RETURN diameter

ALGORITHM DP(u, parent)              // returns h(u)
    max1 = 0, max2 = 0               // two largest (h(child) + 1)
    FOR EACH v IN adj[u] DO
        IF v != parent THEN
            t = DP(v, u) + 1
            IF t > max1 THEN max2 = max1, max1 = t
            ELSE IF t > max2 THEN max2 = t
    diameter = MAX(diameter, max1 + max2)
    RETURN max1                      // h(u)
```

Table 4.3: Trace of the Tree DP Algorithm on the Example Tree (post-order)

Node u	Child values (h(c) + 1)	max1	max2	h(u)	Candidate max1 + max2
8	–	0	0	0	0
7	1 (from 8)	1	0	1	1
5	2 (from 7)	2	0	2	2
4	–	0	0	0	0
2	1 (from 4), 3 (from 5)	3	1	3	4
6	–	0	0	0	0
3	1 (from 6)	1	0	1	1
1	4 (from 2), 2 (from 3)	4	2	4	6

The maximum candidate is 6, found at node 1, so the DP method also gives a diameter of **6 edges**, which agrees with the Two-DFS result.

Complexity. Every node is processed once and every edge is traversed once, giving O(N) time and O(N) space.

4.7 Module: Comparison and Network Interpretation

```
ALGORITHM CompareAndInterpret(dTwoDFS, dDP, a, b)
```

```
    IF dTwoDFS == dDP THEN
        PRINT "Both algorithms agree: diameter = " + dTwoDFS
    ELSE
        PRINT "Mismatch detected"
    PRINT "Longest communication route: " + a + " to " + b
    PRINT "Number of communication hops: " + dTwoDFS
```

In the communication-network model, the diameter gives the largest number of hops that a message needs between any two stations. For the example tree, the two most distant stations are 8 and 6 and the worst-case hop count is 6. The program additionally prints a suggested central server node; for the six-node tree of Chapter 5 this was node 1.

4.8 Comparison of the Two Approaches

Table 4.4: Complexity Summary of the Implemented Operations

Operation	Time	Space	Remarks
Build adjacency list	O(N)	O(N)	2(N − 1) entries
Validation	O(N)	O(N)	Edge count + connectivity
Degree & leaf count	O(N)	O(1) extra	Uses neighbour-list sizes
Tree height	O(N)	O(N)	One DFS
Two-DFS diameter	O(N)	O(N)	Two traversals; easily gives the end-points
Tree DP diameter	O(N)	O(N)	One traversal; uses subtree heights

Both methods have the same asymptotic cost. Two-DFS is conceptually simple and directly yields the two end-points of the longest route, but it needs two traversals. Tree DP needs only one traversal and generalises more easily to other tree problems, but the path itself must be reconstructed separately if it is required. Using both therefore provides a useful consistency check.

The eight-node tree used in this chapter is a hand-derived illustration, not program output. The output that was actually captured is for a different, six-node tree and is presented in Chapter 5. The pseudocode is written at algorithm level; whether the Java source uses recursive or iterative traversal is.

5. RESULTS

This chapter records the testing of the Java program. The "Expected Result" column is derived from the definitions by hand. The "Actual Result" column records only what was actually observed: so far, a single six-node valid tree (Test 9) has been executed and its output captured in Figures 5.1–5.4. All eight originally planned cases remain "Not tested".

Table 5.1: Test Cases and Results

No.	Test Case / Input	Expected Result	Actual Result	Status
1	Example tree: 8 nodes, edges 1–2, 1–3, 2–4, 2–5, 3–6, 5–7, 7–8	Diameter = 6 (8 → 7 → 5 → 2 → 1 → 3 → 6); 3 leaves; height 4 from root 1; both algorithms agree	Not tested	Not tested
2	Path tree: 5 nodes in a line, 1–2–3–4–5	Diameter = 4; end-points 1 and 5	Not tested	Not tested
3	Star tree: centre 1 with 4 leaves (2, 3, 4, 5)	Diameter = 2; 4 leaves; degree of node 1 is 4	Not tested	Not tested
4	Single edge: 2 nodes, edge 1–2	Diameter = 1	Not tested	Not tested
5	Single node (N = 1, no edges)	Diameter = 0 (if supported)	Not tested	Not tested
6	Invalid: number of edges ≠ N − 1	Rejected with an error message	Not tested	Not tested
7	Invalid: graph with a cycle (e.g., 1–2, 2–3, 3–1, 3–4 with N = 4)	Rejected as not a tree	Not tested	Not tested
8	Invalid: disconnected graph (e.g., N = 4, edges 1–2, 1–2, 3–4)	Rejected as not a tree	Not tested	Not tested
9	Six-node tree: edges 1–2, 1–3, 2–4, 2–5, 3–6 (root 1)	Valid tree; 3 leaves; height 2 from root 1; diameter = 4 (4 → 2 → 1 → 3 → 6); both algorithms agree	Valid-tree message; degrees displayed; 3 leaves; height 2; Two-DFS and Tree DP both gave diameter 4 (4 → 2 → 1 → 3 → 6); agreement confirmed; end-points 4 and 6; 4 hops; central server node 1 (Figures 5.1–5.4)	Passed

5.1 Program Output Screenshots

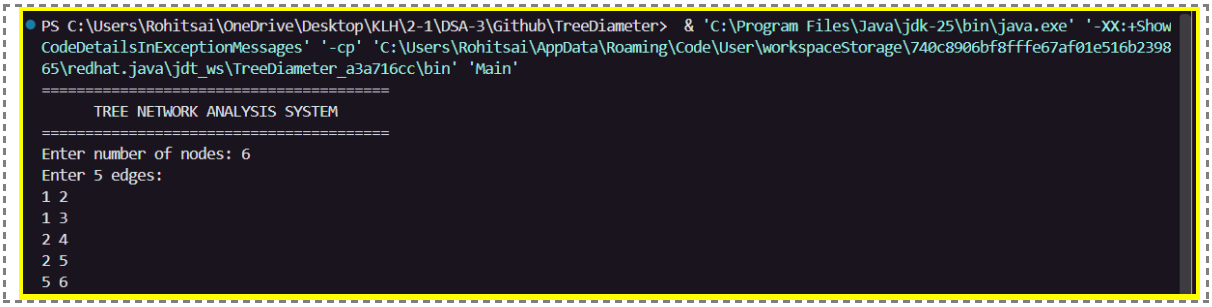


Figure 5.1: Program Start-up and Tree Input Screen

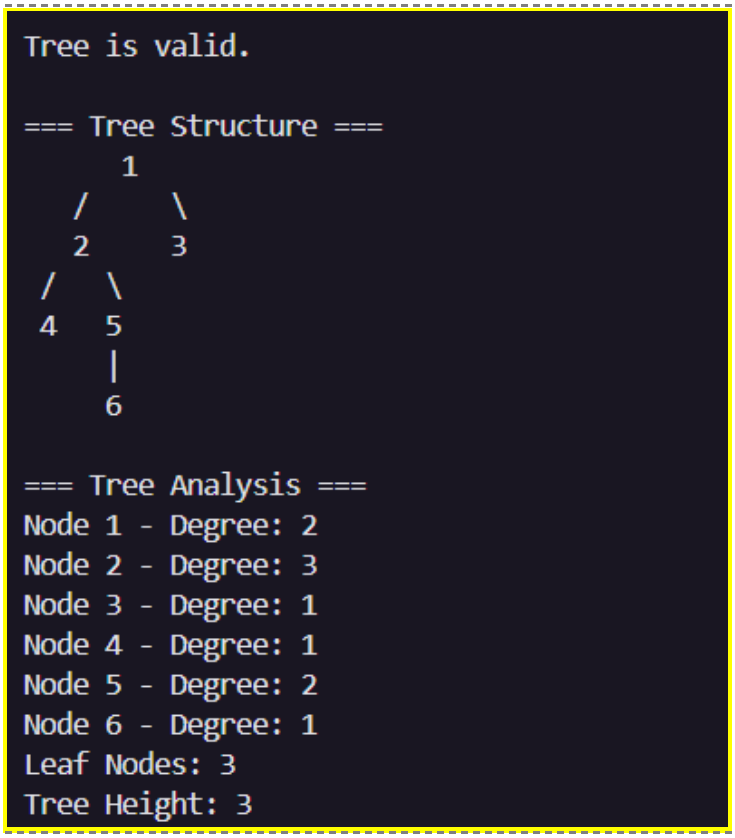


Figure 5.2: Tree Display, Degree, Leaf and Height Output

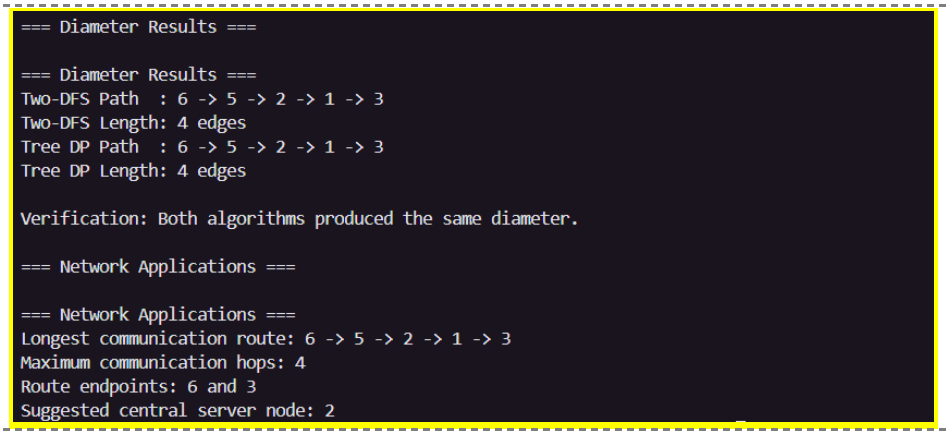


Figure 5.3: Two-DFS and Tree DP Diameter Output with Comparison

```
=== Network Applications ===
Longest communication route: 6 -> 5 -> 2 -> 1 -> 3
Maximum communication hops: 4
Route endpoints: 6 and 3
Suggested central server node: 2
PS C:\Users\Rohitsai\OneDrive\Desktop\KLH\2-1\DSA-3\Github\TreeDiameter>
```

Figure 5.4: Communication Route Interpretation Output

5.2 Discussion of Results

Test 9, the six-node tree with edges 1–2, 1–3, 2–4, 2–5 and 3–6, was executed and its output captured (Figures 5.1–5.4). The program accepted the input, reported a valid tree, printed the tree structure and displayed the node degrees, a leaf count of 3 and a tree height of 2.

Both diameter algorithms returned a diameter of 4 edges along the path 4 → 2 → 1 → 3 → 6, and the program's verification message confirmed that the two results are the same. The longest communication route was reported as 4 → 2 → 1 → 3 → 6, with 4 hops between end-points 4 and 6, and node 1 was suggested as the central server, which is the middle node of that route. These values agree with the expected results derived by hand. The tree has more than one longest path (5 → 2 → 1 → 3 → 6 also has 4 edges); the program reported one of them, which is consistent because all longest paths have the same length.

Only this one case has been executed. The eight originally planned cases in Table 5.1 (the eight-node example, path, star, single-edge, single-node, invalid edge count, cycle and disconnected graph) are marked "Not tested", so this report makes no claim about the program's behaviour on those inputs, including its handling of invalid input. A single successful run shows that the implementation works for this input; it is not proof of correctness for all trees.

6. CHALLENGES, LIMITATIONS AND SOLUTIONS

6.1 Issues Faced During Development

- **Input validation:** Distinguishing the different ways in which an input can fail to be a tree (wrong edge count, duplicate edges, cycles, disconnected components) has to be handled carefully.
- **Recursion depth:** For a path-like tree with a very large number of nodes, a recursive DFS may exceed Java's default stack depth.
- **Consistency between the two algorithms:** Handling the root and the parent argument correctly in the DP method so that it agrees with Two-DFS. 6.2 Solutions Applied
- Validated the number of edges and checked connectivity with a traversal before any analysis was run.Used a parent argument in the DFS to avoid revisiting nodes.
- The program itself compares the results of both algorithms and reports whether they agree (demonstrated in Figure 5.3), which helps to detect implementation errors.

6.3 Limitations of the System

- **Unweighted trees only:** Every edge counts as one hop; edge weights (e.g., link latency) are not considered.
- **Console interface:** The interface is command-line based, with no graphical display of the tree.
- **In-memory processing:** The tree is processed in memory and is not stored in a database.
- **No real-time simulation:** The system analyses the structure of the network but does not simulate message traffic.
- **Limited testing:** Only one test case (the six-node valid tree) has been executed and documented; the remaining planned cases, including invalid-input handling, are untested.

7. CONCLUSION AND FUTURE WORK

7.1 Conclusion

The Tree Network Analysis and Optimization System shows how fundamental DSA-3 concepts can be applied to the analysis of a communication network modelled as a tree. The program builds a tree from user input, represents it with an adjacency list, validates it, reports its basic structural properties, and calculates the diameter by two different linear-time algorithms, Two-DFS and Tree Dynamic Programming. Comparing the results of the two methods provides a reliable cross-check, and presenting the diameter as the longest communication route links the algorithmic result to a practical networking interpretation. For the six-node tree executed during testing, both algorithms reported a diameter of four edges ($4 \rightarrow 2 \rightarrow 1 \rightarrow 3 \rightarrow 6$), the program confirmed that the two results agree, and it suggested node 1 as the central server. This is a single demonstrated case; the remaining planned test cases have not yet been executed, so no general claim of correctness is made.

7.2 Future Work

- **Weighted trees:** Extend the algorithms to edges with weights such as latency or distance.
- **Complete testing:** Execute and document the remaining planned cases in Table 5.1 (path, star, single-edge, single-node and the invalid inputs).
- **Graphical interface:** Build a visual tool that draws the tree and highlights the longest route.
- **File-based input and larger datasets:** Read large trees from files and measure the running time experimentally.
- **Iterative DFS:** Replace recursion with an explicit stack to handle very deep trees.

7.3 Skills Learned

- Representation of graphs and trees using adjacency lists.
- Depth-First Search and its use for computing distances in trees.
- Formulating and solving a tree problem using dynamic programming.
- Analysing time and space complexity.
- Designing a modular Java program and testing it with a sample tree.

REFERENCES

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed., MIT Press, 2009.

[2] E. Horowitz, S. Sahni, and S. Rajasekaran, *Fundamentals of Computer Algorithms*, 2nd ed., Silicon Press, 2008.

[3] J. Kleinberg and É. Tardos, *Algorithm Design*, Pearson, 2006.

[4] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed., Addison-Wesley, 2011.

[5] Oracle, “Java SE Documentation”. Available: https://docs.oracle.com/en/java/